```
  content: "Hey! I'm pretty new here! Glad to be here! This is my
first time here, so please be nice to me (and each other).")
```

Once our web application structure is adequately prepared, we'll pick a simple rule to implement: **A user can delete his/her own posts; he/she cannot delete posts created by other users**. The following four fundamental steps will make CanCan handle authorization work for you.

7. Defining authorization rules: All privilege rules reside in `ability.rb`, which is placed under `app/models`. Run `rails g cancan:ability` so that Rails generates `ability.rb` for you. CanCan provides you with `can` and `cannot` methods for defining access controls. Let's take a look at an example:

```
# app/models/ability.rb
class Ability
  include CanCan::Ability

  def initialize(user)
    if user && user.persisted? # Logged-in user
      # User can destroy his/her own post
      can :destroy, Post, user_id: user.id

      # Non-logged in users cannot destroy Posts.
      # Typically, can is used a lot more than cannot.
      # cannot :destroy, Post, user_id: nil
    end
  end
end
```

As we have seen, the `can` and `cannot` methods take three arguments: action name, the resource class, and a hash of rule conditions. Although the arguments seem restrictive, the methods can actually accept different kinds of arguments such as SQL conditions and blocks. We will discuss this in detail in a later section.

8. Restricting views based on the rules: Now that we have the access rules, we can apply them in views. CanCan comes with view helpers (`can?` and `cannot?`) to check authorization in views.

```
<%# app/views/posts/index.html.erb %>
<% if can?(:destroy, post) %>
  <%= link_to('Del', post, class: 'btn-gray thinner-padding',
method: :delete, data: { confirm: 'Are you sure?' }) %>
<% end %>
```


www.it-ebooks.info